CS-GY6803 - ISSEM - Lab 1

For labs in this class, we will be using Jupyter notebooks which is a useful application for writing and compiling Python code. It contains all the necessary libraries and packages which can be used for labs and projects for this class.

Setup

-

Install the Jupyter notebook. You can install in one of the following ways:

- Refer to link: https://jupyter.org/install
- Alternatively, you can install Anaconda using below link in your system. Based on the OS you have, please select appropriate version. https://www.anaconda.com/products/individual-d

-

Post installation, register into the anaconda to access Jupyter notebook.

-

After installing and registering on Anaconda, create a new Python 3 notebook.

Initialize

In the first cell, put the heading as Lab1 using Heading option in cell type.

Also, in the first cell, change the cell type to Markdown and write the name of all group members and their Net IDs under the heading.

Create a new cell (type- code) and write the following code snippet and run the cell, adding in your group number.

Understanding Python

i. **What operation is the following piece of code performing?**

Use comments to explain the relevant parts of the code.

```
text = "Was it a rat I saw"

cleaned_text = ""

for char in text:
    if char.isalnum():
        cleaned_text += char.lower()

reversed_text = cleaned_text[::-1]

if cleaned_text == reversed_text:
```

```
    result = "Palindrome"
else:
    result = "Not a Palindrome"

print(result)
```

ii. Consider the following piece of code.

Explain each part of the code using comments. Use the same logic to check whether the following inputs are suspicious or not:

"admin"
"root;drop table users"
"hello123"
"user'='user"
"safeInput"

```
user_input = "admin' OR '1'='1"

suspicious = False

for char in user_input:
    if char in ["'", ";", "="]:
        suspicious = True

if suspicious:
    print("Suspicious input detected")
else:
    print("Input looks safe")
```

Basic Cybersecurity

Please explain the following terms.

1. Authentication
2. Authorization
3. Confidentiality
4. Encryption

5. Cipher text
6. Hashing
7. Double Strength Encryption
8. Digital Signature

Assignment: Implementing the RSA Algorithm

RSA (Rivest-Shamir-Adleman) is a foundational algorithm for modern digital security. RSA is an **asymmetric** system. It uses a **Public Key** to lock data and a **Private Key** to unlock it.

The Mathematical Foundation

The security of RSA relies on the fact that it is easy to multiply two large prime numbers together, but extremely difficult to factor the result back into the original primes.

To implement RSA, we use three variables:

- **n (The Modulus):** The product of two primes p and q.
- **e (Encryption Exponent):** A number used for the public key.
- **d (Decryption Exponent):** A number used for the private key.

Encryption and Decryption Formulas

To encrypt a numerical message M into ciphertext C:

C = M^e mod n

To decrypt the ciphertext C back into the original message M:

M = C^d mod n

Note on Python: When calculating large exponents with a modulus, using (M ** e) % n is inefficient and can cause memory errors. Instead, use Python's built-in 3-argument power function: pow(base, exponent, modulus).

Implementation Task

Your goal is to write a flexible RSA implementation. Write two functions that accept the keys as parameters:

1. **rsa_encrypt(message, e, n)**:
   - **Input**: An integer message, an encryption exponent e, and a modulus n.
   - **Output**: The resulting integer ciphertext.
2. **rsa_decrypt(ciphertext, d, n)**:
   - **Input**: An integer ciphertext, a decryption exponent d, and a modulus n.
   - **Output**: The original integer message.

Testing Your Code

Use the table below to verify that your functions work with different key sets.

Requirements:

- Handle only integer inputs.

- The functions must work for any valid e, d, n provided.

Key Set Name	Primes (p,q)	Modulus (n)	Public Exponent (e)	Private Exponent (d)	Plaintext (M)	Expected Ciphertext (C)
Set A	3, 11	33	3	7	**15**	9
Set B	5, 11	55	3	27	**42**	3
Set C	7, 13	91	5	29	**10**	82

•

Hashing

What is hash function in cryptography?

Write a hash function in Python to create the hash value of below message:

∘

message = "Information Systems Security Engineering and Management Summer 2026"

∘

To hash the above function use the Python library hashlib (SHA256)

Hash verification:

Data can be compared to a hash value to confirm its integrity.

The data is hashed at a certain time and the hash value is protected in some way (for this lab, you can note it down). Later, the data can be hashed again and compared to the protected value. If the hash values match, the data has not been altered. If the values do not match, the data has been corrupted.

Write a Python function which implements hash_verification(data, original_hash) as explained above. Use that function to verify the data integrity of the message in the beginning of this section.

Submission

This is a group submission. Please submit only one python notebook per group per part

of the lab. Make sure to run all the functions and then download the notebook in a .ipynb format. Title of the notebook should be lab1_group<group_number>.ipynb. Good luck!